



Fachhochschul-Bachelorstudiengang
SOFTWARE ENGINEERING
A-4232 Hagenberg, Austria

Creating a Visual Studio Extension to support the development of Discord Bots with Discord.Net

(Version 1)

Bachelorarbeit

zur Erlangung des akademischen Grades
Bachelor of Science in Engineering

Eingereicht von

Stefan Christian Merwa

Begutachtet von FH-Prof. DI Johann Heinzelreiter

Hagenberg, Februar 2024

Declaration

I hereby declare and confirm that this thesis is entirely the result of my own original work. Where other sources of information have been used, they have been indicated as such and properly acknowledged. I further declare that this or similar work has not been submitted for credit elsewhere.

This printed thesis is identical with the electronic version submitted.

Date

Signature

Contents

1 Abstract	4
2 Kurzfassung	5
3 Introduction	7
4 Discord	8
4.1 Overview	8
4.2 Discord Bots	8
4.3 Discord.Net	9
5 Architecture	10
5.1 Overview	10
5.2 Visual Studio Extension Development	11
5.2.1 Anatomy	12
5.2.2 Commands	14
5.2.3 Displaying Custom Windows	17
5.2.4 Adding OutputWindowPanels	18
5.3 MS Roslyn	18
5.3.1 Compiler phases	19
5.3.2 API layers	19
5.3.3 Code Analysis	20
5.3.4 Syntax Visualizer in Visual Studio	26
5.4 Editor API	27
5.5 Managed Extensibility Framework	27
6 Building the Visual Studio Extension	29
6.1 Creating a new Extension Project	29
6.2 VS Commands	29
6.3 Custom windows	29
6.4 MEF Extension	29
6.5 Code Analyser	29
7 Closing	30

1 Abstract → vor Inhaltsverzeichnis

Developing third party applications for Discord can require repeated coding inputs. Something like a code base for specific needs could be generated or entity ids that can be copied from the Discord App could be provided directly while coding in the Visual Studio. These things can be achieved by creating a Visual Studio Extension that makes use of other various Microsoft technologies like the Editor API and MS Roslyn.

Warum verwenden Sie hier die Möglichkeitsform?
Beschreiben Sie die Anforderungen und die Ergebnisse
Ihre VS-Erweiterung.

2 Kurzfassung

Die Entwicklung von Drittanbieter-Anwendungen für Discord kann mehrfache Code-Wiederholung erfordern. So könnte z. B. eine Codebasis für bestimmte Anforderungen generiert werden oder Entity-Ids, die normalerweise manuell aus Discord kopiert werden, könnten direkt während des Programmiervorgangs in Visual Studio bereitgestellt werden. Diese Dinge können durch die Erstellung einer Visual Studio-Extension erreicht werden, welche andere verschiedene Microsoft-Technologien wie die Editor-API und MS Roslyn nutzt.

Anforderungen, Funktionen, Funktionalitäten o. Ähnliches

List of Figures

1	Workflow overview to acquire the wanted solution	10
2	Visual Studio Installer	11
3	File structure of a VSIX Project	12
4	Design view of the .vsixmanifest file	12
5	.vsixmanifest file assets	13
6	Custom output window pane	18
7	Compiler pipeline functional areas	19
8	Workspace relation to host environment and editing	21
9	Syntax Visualizer	27

Listings

1	Command to retrieve data from Discord API	14
2	Command placement	14
3	Possible command base	15
4	Base class for async command handling	15
5	Custom command	16
6	Calling commands InitializeAsync in package initialization	17
7	Displaying a custom window	17
8	Displaying a custom window	18
9	Roslyn Workspace	21
10	Syntax Example	24
11	Semantics Example	26
12	ICustomHandler for MEF example	28
13	CustomHandler Export for MEF example	28
14	ICustomHandler Import for MEF example	28

3 Introduction

Gesamteindruck:

- Rechtschreibung: okay
- Qualität Text: okay. Sehr gutes Englisch.
- Inhalt:
 - Der Abschnitt zu Discord und vor allem jener zu VS-Extensions gefallen mir sehr gut.
 - Der Abschnitt zu Roslyn ist meinem Empfinden nach zu ausführlich gehalten und im Stile eines Referenzhandbuchs geschrieben. Es ist dem Leser sicherlich auch nicht klar, welche Rolle Roslyn für Ihre Arbeit spielt. Versuchen Sie auch Roslyn so vorzustellen wie die VS-Erweiterungen, nämlich aus der Anwenderperspektive. Der Leser soll ein Gefühl für die wesentlichen Konzepte bekommen und muss nicht über alle APIs informiert werden. Sie haben sich sehr stark an der Dokumentation orientiert.
 - Sie sollten für das Kapitel „Architektur“ einen besseren Titel finden („Foundations“ oder etwas Ähnliches).
- Literatur: siehe Anmerkungen im entsprechenden Abschnitt.

4 Discord

4.1 Overview

Fügen Sie zwischen Überschriften immer Text ein, der eine Einführung in die Thematik enthält oder einen Überblick über die Unterabschnitte gibt. Hier könnte man z.B. die Überschrift 4.1 reyn lassen.

Discord, a by now popular voice, video, and text chat app, has significantly influenced the way people around the world communicate, especially within various communities and friend groups. Originally developed as a tool for gamers, it has moved on from its initial purpose to more a versatile platform for different conversations, ranging from casual chats to more structured discussions (Discord, 2022).

Origin

The application was founded by Jason Citron and Stanislav Vishnevskiy, who shared a passion for gaming and a vision for improved online communication. Citron, an American entrepreneur, founded OpenFeint, an early mobile gaming platform, and Vishnevskiy, a Russian software engineer, had a background in projects like Google Hangouts and Yahoo! Messenger. Their combined expertise in software engineering and game design facilitated the creation of Discord, launched in May 2015 (TheEnlightenedMindset, 2023).

Architecture and Features

Discord is renowned for its server-based structure, which forms the core of its functionality. These servers, customizable as public or private consist of voice and text channels. This structure enables many different communities, from study groups and book clubs to remote workers, to establish their purpose within the platform. Another significant aspect of Discord is its comprehensive moderation capabilities. Server administrators have the authority to manage user permissions, establish automated moderation protocols, and utilize features like word filtering (whitelist or blacklist) to maintain a safe and respectful environment for community interactions. Furthermore, Discord's ability to integrate smoothly with other well known platforms contributes to its appeal and effectiveness. This integration capability allows users to perform a variety of actions, such as streaming live gaming sessions on Twitch, sharing YouTube videos directly in chat channels, or collaborating on documents via Google Docs. These integrations play an important, if not the most important role in enhancing the overall utility and user experience of Discord (GameInfoHub, 2023).

4.2 Discord Bots

Bots are a key feature of Discord and a type of software that automates tasks and interacts with users in Discord. They are integrated into servers (4.1) and perform a variety of functions based on the commands and services like moderation, games, music, internet searches, and payment processing based on the programming they are given (Agnihotri et al., 2022). Bots use Discord's Application Programming Interface to

interact with the server and its users. This API allows bots to send and receive messages, manage user roles, and more. It is important to note, that one bot can be used for as many servers ^{as ?} required.

Preparation

Discord bot development is a multifaceted process involving understanding the Discord platform, using its APIs, managing bot accounts, and writing and troubleshooting code. Developers must consider aspects like bot complexity, security, permissions, and community engagement. Utilizing available resources, best practices, and community support is key to creating successful and effective Discord bots (ToptalDevelopers, 2022). Before developing the bot, it first needs to be created through the Discord Developer Portal. There is an option to create a new application, which then will contain the bot user that can be configured with various settings like permissions and also provides a token for establishing a connection to Discord's API.

Development

Developing a Discord bot involves choosing a programming language and corresponding library that interfaces with the Discord API. Each language offers unique libraries or frameworks, designed to simplify the bot development process by abstracting the Discord API's complexities. To name a few, amongst the most popular ones are *Discord.js* for Javascript, *discord.py* for Python, *JDA* for Java, *DiscordGo* for Go and *Discord.Net* for C# . The choice of language and library depends on the developer's familiarity with the language, the specific requirements of the bot, and the desired level of abstraction from the Discord API. Each language and library has its strengths and is suited to different types of projects and developer preferences.

4.3 Discord.Net

Discord.Net is an asynchronous, multi-platform .NET Library [?] 4.2 designed to interface with the Discord API. It is an unofficial .NET API wrapper for the Discord client and also very feature rich by supporting slash commands, interaction frameworks and message components. To get started with Discord.Net, a good understanding of advanced .NET programming concepts like the Task-based Asynchronous Pattern ^{is required ?} since the library extensively uses them. There are a lot of tutorials and also a comprehensive guide that provides detailed instructions and examples to help developers (Discord.Net, 2015). There exists a public Discord server for Discord.Net with more than 3000 current members, some of whom are willing to offer support and answer questions about the library. For common C# questions there also exists a Discord server with almost 50.000 members, where there is a channel dedicated to Discord development.

5 Architecture

5.1 Overview

To achieve the wanted capabilities for the Visual Studio Extension, multiple technologies are required. First a core understanding of how Visual Studio Extensions are developed and what features and possibilities they bring to the table is necessary. To effectively implement this process, it's essential to first set up a way to retrieve data from the Discord API. Since it's common to develop just one bot per C# project, it makes sense to trigger the process for each individual project. To make this happen, a command can be made available by simply right clicking on the project. This data is then used for code completion via the Editor API. Additionally, using the said API requires working with the Managed Extensibility Framework (MEF), since the Editor API depends on it. An important step in this process is figuring out exactly when and what type of data should be made available for code completion, and this is where MS Roslyn comes in, offering precise code analysis to make these determinations.

Unklar. Meinem Sie hier, dass das Kontext Menü geöffnet wird? Ev. Screenshot einfügen.

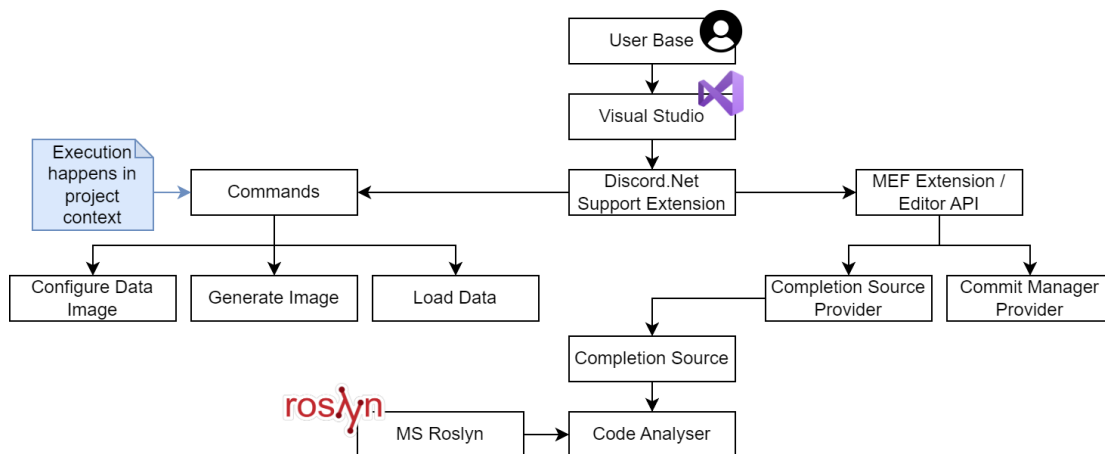


Figure 1: Workflow overview to acquire the wanted solution

To clarify this workflow in a more detailed manner, imagine a user who is coding in Visual Studio and has installed an extension that provides the necessary features. This extension includes various Commands that can be accessed by right clicking on a project. For interacting with the Editor API, a separate extension project is created and then added to the main project as a MEF component. In this part of the project, the user's code is analyzed using MS Roslyn. The finer points of these steps are thoroughly explained in the following sections.

5.2 Visual Studio Extension Development

To participate in Visual Studio Extensibility (VSIX), the workload needs to be installed.

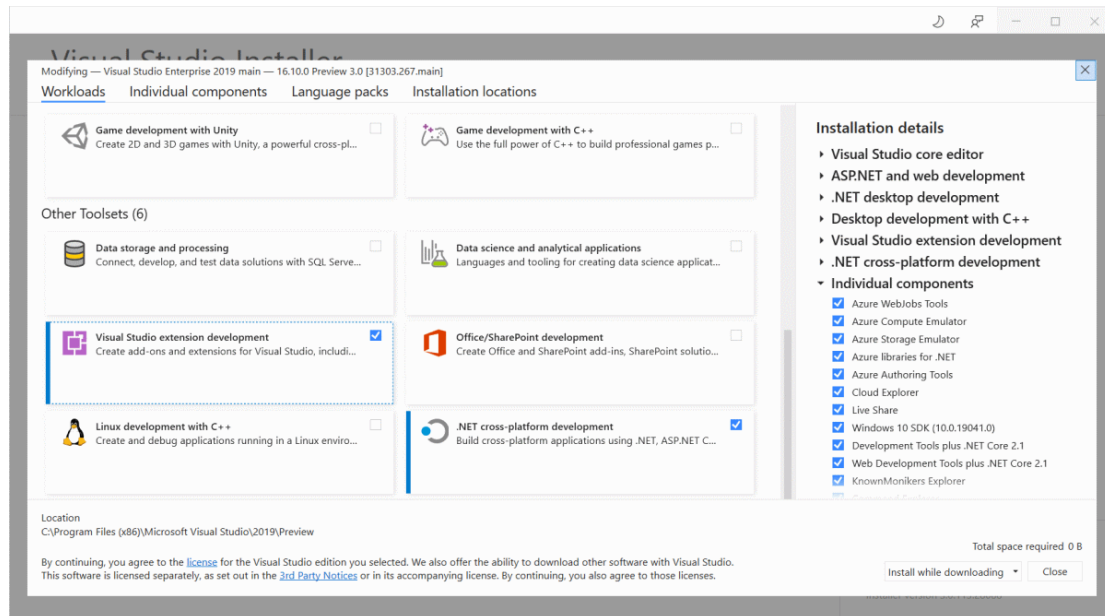


Figure 2: Visual Studio Installer¹

The types of extensions you can create are quite varied. They can range from supporting new languages not included in Visual Studio, to productivity tools that enhance the core IDE experience, to domain-specific designers for specific scenarios like data design or cloud support. In Visual Studio, you can extend various parts like menus, toolbars, commands, windows, solutions, and projects. However, the most common features extended are commands, menus, toolbars, windows, IntelliSense, and projects (Microsoft, 2023). For examples, there exists the Visual Studio Marketplace, where many Extensions are available for free or are even open sourced.

¹Source: <https://www.vsixcookbook.com/assets/img/vs-installer.png> (2024-01-07)

5.2.1 Anatomy

File Structure

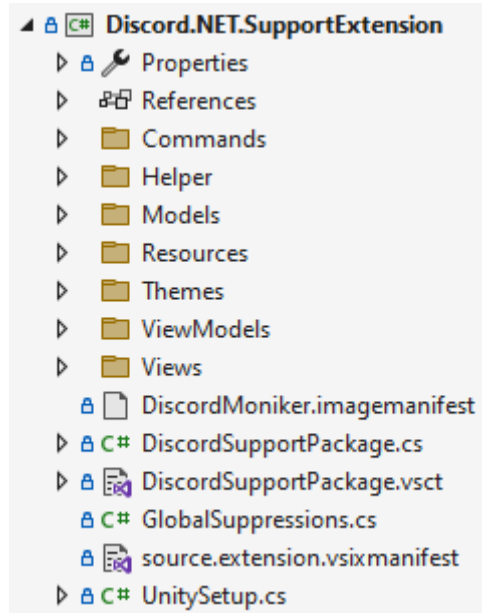


Figure 3: File structure of a VSIX Project

The primary file in a Visual Studio extension project is the `.vsixmanifest` file (source.extension.vsixmanifest in figure 3), an XML file that provides essential information about the extension to Visual Studio. It registers all the components of the extension and is the only required file in a VSIX project.

*immer probieren,
auch chapter x,
Section y etc.*

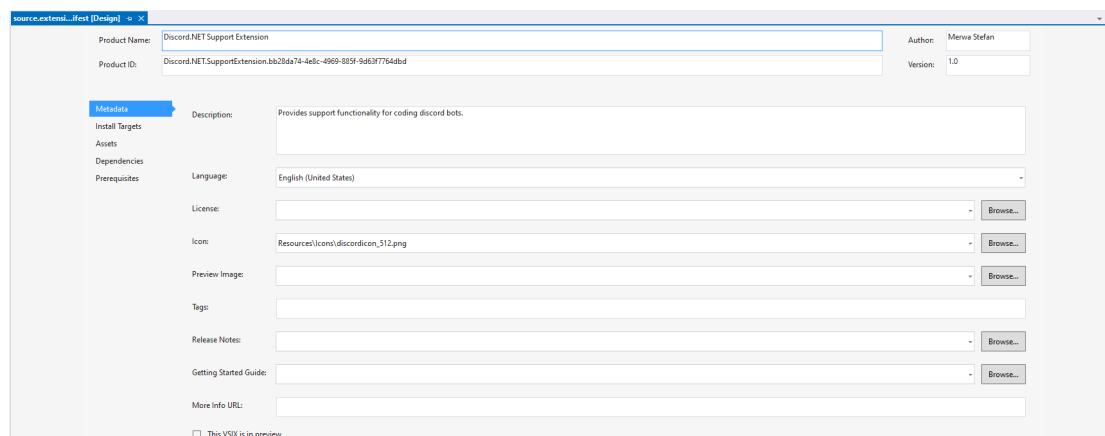
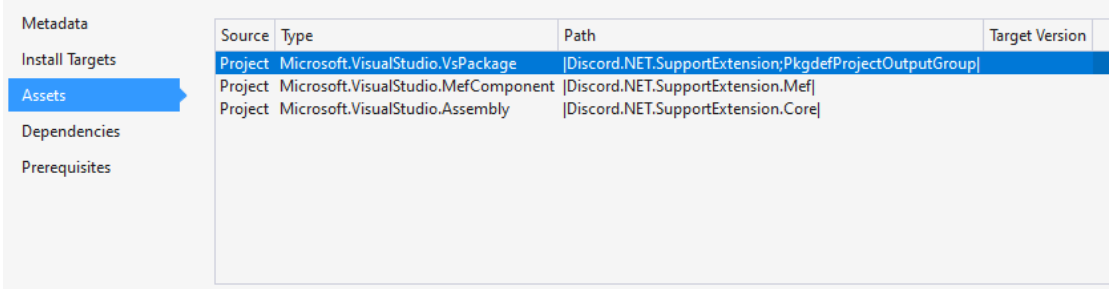


Figure 4: Design view of the .vsixmanifest file

As mentioned in 5.1 it is required to import the second extension as MEF-component. So a new asset is required, that goes with the source, type and path, as in this case the source is a project, the type is a MefComponent and the Path refers to the second extension project *Discord.NET.SupportExtension.Mef* (Microsoft, 2021b).



Source	Type	Path	Target Version
Project	Microsoft.VisualStudio.VsPackage	Discord.NET.SupportExtension;PkgdefProjectOutputGroup	
Project	Microsoft.VisualStudio.MefComponent	Discord.NET.SupportExtension.Mef	
Project	Microsoft.VisualStudio.Assembly	Discord.NET.SupportExtension.Core	

Figure 5: .vsixmanifest file assets

Commands within the extension are defined in the .vsct file (*DiscordSupportPackage.vsct* in figure 3), another XML file. This file includes definitions for button commands, menus, and keyboard shortcuts. Its contents are compiled into a blob within the output .dll, which Visual Studio utilizes to build its command table menu structure. However, this file only declares components and does not manage command executions. The Package.cs file (*DiscordSupportPackage.cs* in figure 3), serves as the primary entry point for most extensions. This file typically contains command handlers, tool windows, options pages, services, and other components (Microsoft, 2021b).

*line .NET-Resource
in Assembly?*

Compilation

The project is compiled into a .vsix file, which can be found in either the /bin/debug or /bin/release folder, depending on the build configuration set for your current solution. The workload for Visual Studio extension development includes specialized MSBuild targets and tasks specifically designed for managing VSIX projects. During the build process of the VSIX project, it is automatically deployed to the Experimental Instance of Visual Studio (Microsoft, 2021b).

Experimental Instance

To protect your main Visual Studio development environment from potential alterations by untested applications, the Visual Studio SDK offers an experimental space for testing and experimentation. This allows you to develop new applications using the regular Visual Studio interface, but they are run in a separate, isolated Experimental Instance. This ensures that your main development environment remains unaffected. Furthermore, every application equipped with a VSIX package automatically launches the Visual Studio Experimental Instance in debug mode when you run the application.

5.2.2 Commands

Definition

Every button in every menu is a command. To add commands to an extension, it has to be defined in the vsct file first:

Listing 1: Command to retrieve data from Discord API

```
1 <Button guid="CommandSet" id="CustomCommand" priority="0x0200" type="
  Button">
2 <Parent guid="GroupSet" id="CSharpProjectMenuGroup" />
3 <Icon guid="discordImages" id="bmpPic1" />
4 <Strings>
5 <ButtonText>Click me</ButtonText>
6 </Strings>
7 </Button>
```

The button will be placed at the Parent object, which is the C# project menu `IDM_VS_CTXT_PROJNODE`:

Listing 2: Command placement

```
1 <Groups>
2 <Group guid="GroupSet" id="CSharpProjectMenuGroup" priority="0x003"
  />
3 </Groups>
4
5 <CommandPlacements>
6 <!-- Project Context -->
7 <CommandPlacement guid="GroupSet" id="CSharpProjectMenuGroup"
  priority="0x004">
8 <Parent guid="guidSHLMainMenu" id="IDM_VS_CTXT_PROJNODE" />
9 </CommandPlacement>
10 </CommandPlacements>
```

ev. Screenshot vom daraus resultierenden Kontextmenü hinzufügen

Execution

Wo sieht man die GUID?

Now that the command is defined in the command table, the invocation logic must be created. Every command is accessed via a GUID and ID from Visual Studio (as seen in listing 1 and listing 2). Keep in mind, that if multiple commands are used, creating a base class might be a wise decision.

Listing 3: Possible command base

```

1 public abstract class CommandBase {
2     protected AsyncPackage Package { get; }
3     protected abstract Guid CommandSet { get; }
4     protected abstract int CommandId { get; }
5
6     protected CommandBase(AsyncPackage package, IMenuCommandService
7         commandService) {
8         this.Package = package ?? throw new ArgumentNullException(nameof(
9             package));
10        if (commandService == null) throw new ArgumentNullException(nameof(
11            commandService));
12
13        CommandID menuCommandId = new CommandID(CommandSet, CommandId);
14        OleMenuCommand menuCommand = new OleMenuCommand(this.Execute,
15            menuCommandId);
16
17        commandService.AddCommand(menuCommand);
18    }
19
20    protected abstract void Execute(object sender, EventArgs e);
21    protected IAsyncServiceProvider AsyncServiceProvider => Package;
22    protected IServiceProvider ServiceProvider => Package;
23 }

```

in Listing 3

Code line 11 shows that the *Execute* Method is passed as *EventHandler* to the menu command, so it cannot be async without losing the execution state. There exists a well known workaround, for which it also makes sense to create a base class.

Listing 4: Base class for async command handling

```

1 public abstract class AsyncCommandBase : CommandBase {
2     private readonly Action<Exception> onException;
3     private bool isRunning = false;
4
5     protected AsyncCommandBase(AsyncPackage package, IMenuCommandService
6         commandService, Action<Exception> onException) : base(package,
7         commandService) {
8         this.onException = onException;
9     }
10
11    protected override async void Execute(object sender, EventArgs e) {
12        try {
13            if (isRunning)

```

```

12         throw new CommandException($"Command {this.GetType().Name} is
           already running.");
13
14         isRunning = true;
15         await ExecuteAsync(sender, e);
16     }
17     catch (Exception ex) {
18         onException?.Invoke(ex);
19     }
20
21     isRunning = false;
22 }
23
24 protected abstract Task ExecuteAsync(object sender, EventArgs e);
25 }

```

This is also known as RelayCommand. *Async methods with void as return type* Async voids are usually a very bad practice, because the execution results in *Fire and Forget*. This workaround provides a Delegate that will be invoked if an exception occurs while command execution. After creating these classes, the final command class can look like this.

Listing 5: Custom command

```

1 internal sealed class CustomCommand : AsyncCommandBase {
2     protected override Guid CommandSet => PackageGuids.CommandSet;
3     protected override int CommandId => PackageIds.CustomCommand;
4
5     internal CustomCommand(AsyncPackage package, IMenuCommandService
        commandService, Action<Exception> onException) : base(package,
        commandService, onException) {
6         // Initialization work
7     }
8
9     public static CustomCommand Instance { get; private set; }
10    public static async Task InitializeAsync(AsyncPackage package,
        IMenuCommandService commandService, Action<Exception>
        onException) {
11        Instance = new CustomCommand(package, commandService, onException)
12        ;
13    }
14    protected override async Task ExecuteAsync(object sender, EventArgs
        e) {
15        // Command Execution
16    }
17 }

```

The *PackageGuids* class is generated from the .vsct file, so it provides the required GUIDs and IDs for all defined commands.

One last important thing is to call the Initialization method *InitializeAsync* from inside the package class (*DiscordSupportPackage.cs* from ~~figure~~ 3) to create the Singleton instance of said command:

command's ?

Listing 6: Calling commands *InitializeAsync* in package initialization

```
1 protected override async Task InitializeAsync(CancellationTok
    cancellationToken, IProgress<ServiceProgressData> progress) {
2     IMenuCommandService commandService = await this.GetServiceAsync(
        typeof(IMenuCommandService)) as IMenuCommandService;
3     await CustomCommand.InitializeAsync(this, commandService,
        OnException);
4 }
```

(Microsoft, 2021b) *← Wo gehört das hin?*

5.2.3 Displaying Custom Windows

** Hinweis, wofür das notwendig ist.*

The IVsUIShell provides functionality to display ~~a~~ windows

Listing 7: Displaying a custom window

```
1 public static void Show(System.Windows.Window window) {
2     // Is required since the Visual Studio Shell should only be used on
        the main thread
3     ThreadHelper.ThrowIfNotOnUIThread();
4
5     IVsUIShell uiShell = (IVsUIShell)Package.GetGlobalService(typeof(
        SVsUIShell));
6     Assumes.Present(uiShell);
7
8     uiShell.GetDialogOwnerHwnd(out IntPtr hwnd);
9     WindowHelper.ShowModal(window, hwnd);
10 }
```

Überlegen Sie, ob Sie nicht generell Klassen-, Methodennamen etc. in einem anderen Font darstellen (z.B. Texttt). Das überlasse ich Ihnen.

5.2.4 Adding OutputWindowPanes

Abgrenzung zu Custom Windows

An OutputWindowPane is useful if the user should get notified about something that happens inside the installed extension. First the pane needs to be created and can then be accessed by loggers to provide logs to the created output window pane.

Listing 8: Displaying a custom window

```
1 private static IVsOutputWindowPane pane;
2 public static void InitOutputLog(string paneName) {
3     ThreadHelper.ThrowIfNotOnUIThread();
4
5     IVsOutputWindow logWindow = AsyncPackage.GetGlobalService(typeof(
6         SVsOutputWindow)) as IVsOutputWindow;
7     Guid guid = Guid.NewGuid();
8     logWindow.CreatePane(ref guid, paneName, 1, 1);
9     logWindow.GetPane(ref guid, out pane);
10 }
11 public static void OutputWindowFunc(LogStatement obj) {
12     Func<Task> logTask = new Func<Task>(async () => {
13         await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();
14         pane.OutputStringThreadSafe(obj.ToString() + "\n");
15     });
16
17     ThreadHelper.JoinableTaskFactory.Run(logTask);
18 }
```

For instance in the following figure, there exists a new OutputWindowPane called "Discord Support Extension" that provides context specific information.

richtige "Quotes" verwenden

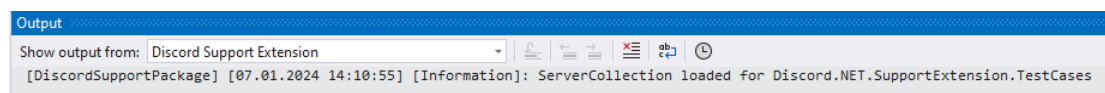


Figure 6: Custom output window pane

5.3 MS Roslyn

← Warum geben Sie dieses Thema in das Kapitel Architektur? Müste das nicht besser im Grundlagenkapitel Platz? & Erläuterung, welche Bedeutung Roslyn für Ihre Arbeit hat.

Microsoft's Roslyn, also known as the .NET Compiler Platform SDK, represents a significant shift in how .NET handles compilation and provides extensive APIs for various compilation phases. Roslyn is essentially a set of services built around two key principles: Extensibility and Maintainability. This design allows for layers above and integration with third party plugins, offering well documented and supported public APIs. External developers can leverage these services to do a variety of innovative things. Some of these can be crafting their own tools tailored to any specific layer of programming tasks or developing simple plugins like diagnostic analyzers, code fixes, refactorings, and IntelliSense providers for certain layers (Vasani, 2017).

*) Bei dieser Art von Zitat beziehen Sie sich auf den letzten Satz od. Gedanken. Wenn sich das Zitat auf den gesamten Absatz beziehen soll, verwenden Sie einen "Topic Sentence". Z.B. "AS started in [...]" (so leiten Sie den Absatz ein).

Das würde auch an anderen Stellen der Arbeit die richtige Wahl sein.

5.3.1 Compiler phases

Roslyn deconstructs the compiler into distinct phases and exposes managed APIs for each phase, making language compilers a service (Developer.com, 2012). The .NET Compiler Platform SDK provides an API layer that mirrors a traditional compiler pipeline for C# and Visual Basic compilers. This pipeline includes four phases. The first phase, parsing, turns the source text into syntax following the language rules. Then, the declaration phase analyzes the source and metadata to form symbols. Following right after, binding, connects identifiers in the code to these symbols. The final phase, emitting, compiles all the information into an assembly, producing IL byte codes. For each of them, the SDK offers a specific object model, allowing access to the processed information (Microsoft, 2021a).

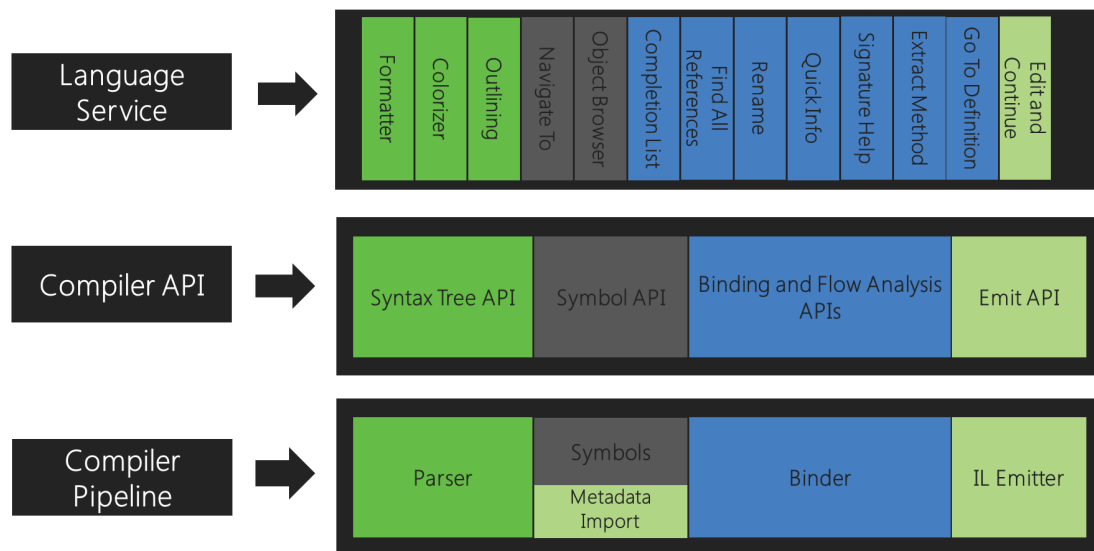


Figure 7: Compiler pipeline functional areas²

5.3.2 API layers

Compiler APIs

They contain object models that represent information exposed at each phase of the compiler pipeline, both syntactic and semantic. They provide an immutable snapshot of a single compiler invocation, including assembly references, compiler options, and source files (Microsoft, 2021a).

Diagnostic APIs

The compiler can produce a set of diagnostics covering syntax, semantics, and assignment errors, as well as warnings and informational diagnostics. These are all exposed

²Source: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/media/compiler-api-model/compiler-pipeline-lang-svc.png> (2024-01-07)

through an extensible API that allows the integration of user defined analyzers into the compilation process (Microsoft, 2021a).

Scripting APIs

Built on top of the compiler layer, these APIs facilitate running code snippets and accumulating a runtime execution context. They are used in scenarios like the C# interactive Read-Evaluate-Print Loop (REPL), allowing C# to be used interactively as a scripting language (Microsoft, 2021a).

Workspace APIs

They assist in organizing information about projects in a solution into a single object model. They offer direct access to the compiler layer object models and are crucial for code analysis and refactoring over entire solutions (Microsoft, 2021a).

5.3.3 Code Analysis

Code analysis with Roslyn is a versatile process making use of workspaces, syntax and semantics to understand and manipulate source code.

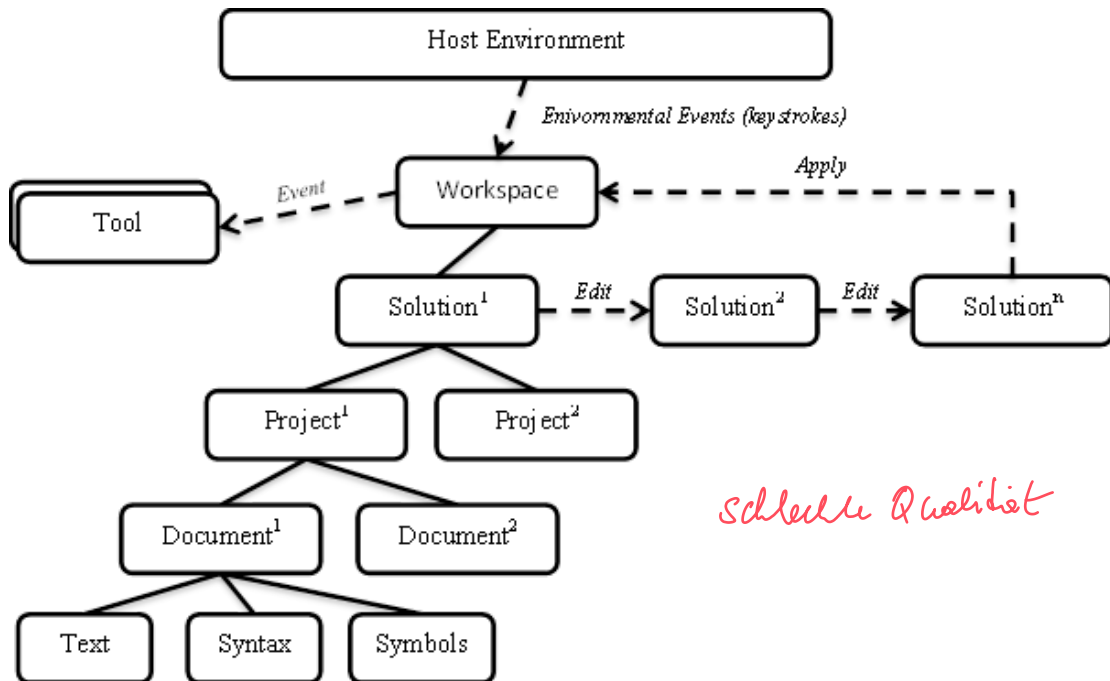
Workspaces

The Workspaces layer in Roslyn is essential for code analysis and refactoring across entire solutions. It organizes project information into a unified object model, allowing easy access to important compiler elements like syntax trees and semantic models, without file parsing or dependency management. Within an IDE, workspaces reflect real-time changes, updating the model as users proceed to make updates like writing code or changing properties (Microsoft, 2021c).

Solutions

Even though workspaces can change with each key press, the solution model can be worked with independently. Solutions are immutable models containing projects and documents, allowing for sharing without conflicts or duplication. Once a solution instance is obtained from the Workspace.CurrentSolution property, it remains unaltered. Modifications to solutions involve creating new instances based on existing ones and applying these changes back to the workspace. Projects within the solution model contain source code documents, parsing and compilation settings, and references, facilitating direct access to compilations. Documents in the model represent individual source files, from which text, syntax tree, and semantic model can be accessed (Microsoft, 2021c).

Satz

Figure 8: Workspace relation to host environment and editing³

Assuming the just mentioned features need to be used in the context of a Visual Studio Extension, the workspace may be accessed like the following example displays.

Listing 9: Roslyn Workspace

```

1 IComponentModel componentModel = (IComponentModel)Package.
  GetGlobalService(typeof(SComponentModel));
2 VisualStudioWorkspace workspace = componentModel.GetService<Microsoft.
  VisualStudio.LanguageServices.VisualStudioWorkspace>();
3
4 Solution solution = workspace.CurrentSolution;
5
6 foreach (Project project in solution.Projects) {
7   Console.WriteLine($"Project: {project.Name}");
8
9   foreach (Document document in project.Documents) {
10    Console.WriteLine($"  \tDocument: {document.Name}");
11  }
12 }
  
```

Syntax Trees

Syntax trees are immutable data structures exposed by the compiler APIs, representing

³Source: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/media/work-with-workspace/workspace-obj-relations.png> (2024-07-01)

the lexical and syntactic structure of source code. Essential for tools like IDEs, code analysis tools, and refactorings, they allow these applications to process and manipulate a project's syntactic structure effectively. They are the primary data structure for numerous tasks including compilation, binding, refactoring, and code analysis, while ensuring that no part of the source code is processed without categorization into defined language elements. They also have three key attributes that include full fidelity, exact text reproduction and immutability and thread safety. Full fidelity means that the syntax tree contains all information found in the source text like grammatical constructs, lexical tokens and everything inbetween like whitespaces and comments. It is possible to get the exact text representation from any syntax node's subtree. This initiates the possibility to use syntax trees as a way to construct and edit code. After a syntax tree is obtained, it's a snapshot of the current code state and does not change under any circumstances. This allows multiple users to interact with the same syntax tree on different threads without locking or duplication. If changes are required, there still exist factory methods to create additional snapshots of the tree. Syntax trees are literal tree structures, where non terminal structural elements parent others. They are also build up on nodes, tokens and trivia (Microsoft, 2021e).

Syntax Nodes

Syntax nodes, a primary element of syntax trees, represent syntactic constructs like declarations, statements and expressions. Each type of node, derived from `Microsoft.CodeAnalysis.SyntaxNode`, is a non-terminal node with children that are either nodes or tokens. These nodes are immutable in nature and maintain a consistent parent since trees are also immutable, with the root node having a null parent. Nodes offer a `SyntaxNode.ChildNodes()` method for sequential child node access and additional methods like `DescendantNodes` to explore the tree's depths. Subclasses of syntax nodes provide specific, strongly typed properties for their children. For instance the `BinaryExpressionSyntax` node includes three properties unique to binary operators consisting of `Left`, `OperatorToken`, and `Right`, where `Left` and `Right` are of `ExpressionSyntax` type, and `OperatorToken` is a `SyntaxToken`. Additionally, some syntax nodes have optional children. In the case of an `IfStatementSyntax` node, there's an optional `ElseClauseSyntax` child, which will return null if it's not included in the node (Microsoft, 2021e).

Syntax Tokens

Syntax tokens are the fundamental elements of language grammar, representing the smallest units like keywords, identifiers, and punctuation. Unlike syntax nodes, they don't parent other nodes or tokens. `SyntaxToken` is a CLR value type for efficiency, with a uniform structure but different properties depending on the token type. For instance, an integer literal token has a `Value` property for the numeric value and a `ValueText` property as a string that accounts for Unicode characters in identifiers, so it shows the actual characters rather than escape sequences in the source text. For example, if the text contains an identifier written as `'\u03C0'`, then the `ValueText` property for this token will return π (Microsoft, 2021e).

Syntax Trivia

Syntax trivia represents the insignificant parts of the source text like whitespaces, comments and preprocessor directives. `SyntaxTrivia` is also a CRL value type and describes all kinds of existing trivia. Since trivia can appear anywhere between any tokens, they are not part of the syntax tree's children. Yet they are important to maintain full fidelity, so they are still accessible by using the `LeadingTrivia` or `TrailingTrivia` property on a `SyntaxToken` (Microsoft, 2021e).

Spans

Each node, token or trivia knows its starting position and the length within the source text. A text position is represented as a zero based character index. Nodes, tokens and trivia contain the properties `Span` and `FullSpan` that are of type `TextSpan`, with `Span` referring to the location without leading or trailing trivia. `TextSpan` simply represents the starting position and the length of its owner and if a `TextSpan`'s length is zero, it refers to a location between two characters (Microsoft, 2021e).

Kinds

Each node, token, or trivia in a syntax tree has a `SyntaxNode.RawKind` property, an integer that specifies its exact syntax element. This value corresponds to language specific enumerations, such as `Microsoft.CodeAnalysis.CSharp.SyntaxKind` for C# , providing a comprehensive list of all syntax elements. The `RawKind` property is important for discrimination between types, especially when they share the same class. For instance, the `BinaryExpressionSyntax` class's `Kind` property identifies whether it's an addition, subtraction, or multiplication expression (Microsoft, 2021e).

Errors

When encountering syntax errors in the source text, the parser still creates a complete syntax tree that can be converted back to the source. Firstly, if a specific token is expected but missing, the parser inserts a 'missing token' in the tree at that location, which has no span and its `IsMissing` property set to true. Secondly, the parser might skip over tokens until it reaches a point where normal parsing can resume. Skipped tokens are then attached as trivia nodes of the type `SkippedTokensTrivia` (Microsoft, 2021e).

Example

Listing 10: Syntax Example

```
1 SyntaxTree tree = CSharpSyntaxTree.ParseText(@"
2 public class CustomClass {
3     public void CustomMethod() {
4     }
5 }");
6
7 CompilationUnitSyntax root = tree.GetRoot() as CompilationUnitSyntax;
8
9 foreach (ClassDeclarationSyntax classDeclaration in root.
10     DescendantNodes().OfType<ClassDeclarationSyntax>()) {
11     Console.WriteLine($"Class: {classDeclaration.Identifier.ValueText}");
12 }
13
14 foreach (MethodDeclarationSyntax methodDeclaration in
15     classDeclaration.DescendantNodes().OfType<
16     MethodDeclarationSyntax>()) {
17     Console.WriteLine($"Method: {methodDeclaration.Identifier.
18         ValueText}");
19 }
20 }
```

A string representing valid C# syntax is used to create a `SyntaxTree`. Then the root is accessed to iterate through the descendant nodes to identify classes and methods, which identifier values are printed to the console. This will result in

```
1 Class: CustomClass
2 Method: CustomMethod
```

Compilation

The information of syntax trees alone does not provide enough information to identify what is being referenced. A name may represent a type, field, method or local variable. Programs can use elements from their own source code and from compiled library files. Since there is no source code and therefore no syntax trees available for assemblies, programs can still refer to elements inside them, and that's where the Semantic model is used. This model also encapsulates the language rules, providing a better understanding of identifiers with their referenced program element Microsoft (2021d).

A compilation represents everything required to compile a C# or Visual basic program, which includes all referenced assemblies, compiler options and source files. The com-

pilation represents each declared type, member or variable as a symbol and contains a variety of methods that support the process of finding symbols that are declared in the source code or imported as metadata from an assembly. Just like syntax trees, compilations are immutable, but a new compilation's creation is supported from an existing compilation, specifying the required changes (Microsoft, 2021d).

Symbols

A symbol represents a distinct element like a namespace, type, method or property that is either declared by the source code or imported from an assembly as metadata. The Compilation type holds methods available to assist in identifying these symbols. For instance, a symbol for a declared type can be found using its metadata name. Each symbol contains additional information determined by the compiler from the source or metadata, including references to other symbols. These symbols are categorized into different types, each represented by a unique interface derived from the ISymbol interface that comes with methods and properties that detail the compiler's findings. For example, the ReturnType property of an IMethodSymbol tells the type of symbol a method returns. Symbols serve as a common representation of namespaces, types and members between source code and metadata. A method declared in the source code and one imported from metadata are both depicted by an IMethodSymbol with the same properties. While symbols share similarities with the CLR type system as represented by the System.Reflection API, they offer a richer modeling of programming language constructs, extending beyond types to include entities like namespaces and local variables. However, symbols primarily represent language concepts rather than CLR concepts. For example, an iterator method in C# or Visual Basic is represented as a single symbol, but when translated to CLR metadata, it becomes a type with multiple methods (Microsoft, 2021d).

Semantic Model

A semantic model represents semantic information for a single source file and is used to discover referenced symbols for a given location, resulting types of any expression and all possible diagnostics (Microsoft, 2021d).

Example

Listing 11: Semantics Example

```

1 SyntaxTree tree = CSharpSyntaxTree.ParseText(@"
2 public class CustomClass {
3     public void CustomMethod(int customParameter) {
4     }
5 }");
6
7 SyntaxNode root = tree.GetRoot();
8 Compilation compilation = CSharpCompilation.Create("CustomCompilation"
9 )
10 .AddReferences(MetadataReference.CreateFromFile(typeof(object).
11     Assembly.Location))
12 .AddSyntaxTrees(tree);
13 SemanticModel model = compilation.GetSemanticModel(tree);
14 MethodDeclarationSyntax methodDeclaration = root.DescendantNodes().
15     OfType<MethodDeclarationSyntax>().First();
16 IMethodSymbol symbol = model.GetDeclaredSymbol(methodDeclaration);
17
18 Console.WriteLine($"Method: {symbol.Name}");
19 foreach (IParameterSymbol parameter in symbol.Parameters) {
20     Console.WriteLine($"Parameter: {parameter.Name}, Type: {parameter.
21         Type}");
22 }

```

Again a string representing valid C# syntax is used to create a SyntaxTree and builds the compilation based on that tree. Then the compilation is used to acquire a SemanticModel, that is then used to get the symbol from the first found MethodDeclarationSyntax. Then all parameters are iterated and its name and type are printed to the console. This will result in

```

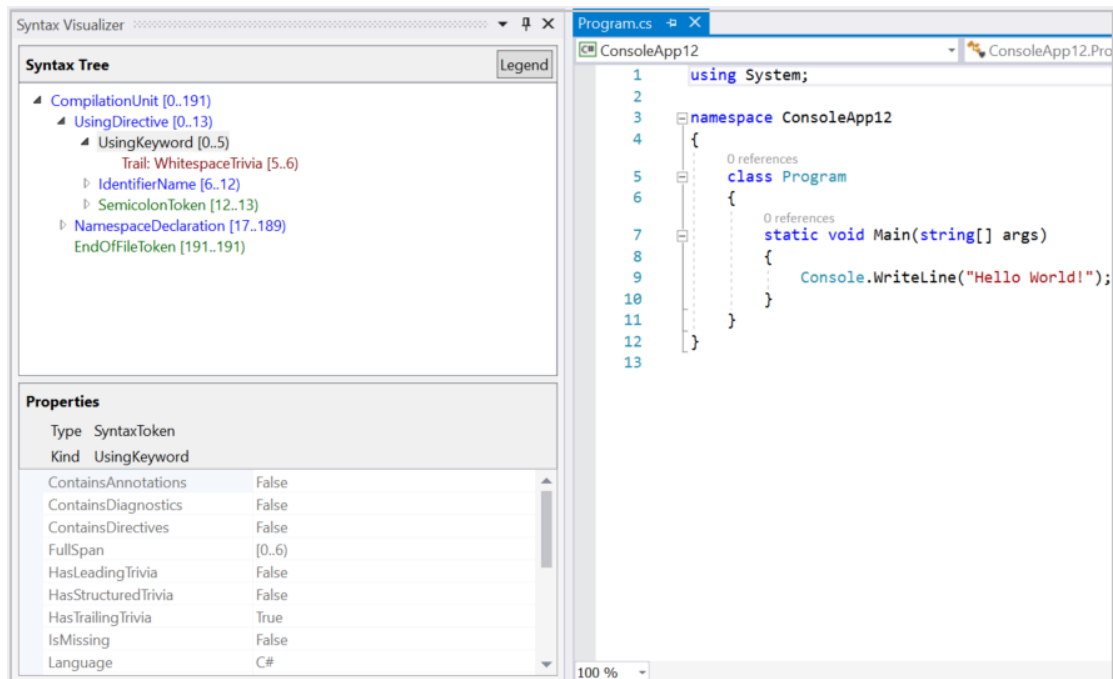
1 Method: CustomMethod
2 Parameter: customParameter, Type: System.Int32

```

quote environment verwenden. Zitat einleiten, besprechen, in Text einbinden

5.3.4 Syntax Visualizer in Visual Studio

"The Syntax Visualizer enables inspection of the syntax tree for the C# or Visual Basic code file in the current active editor window inside the Visual Studio IDE. The visualizer can be launched by clicking on View > Other Windows > Syntax Visualizer. You can also use the Quick Launch toolbar in the upper right corner. Type "syntax", and the command to open the Syntax Visualizer should appear" (Microsoft, 2022)

Figure 9: Syntax Visualizer⁴

5.4 Editor API

The Editor API is used to interact with the Visual Studio Editor.

5.5 Managed Extensibility Framework

The Managed Extensibility Framework (MEF) is a tool designed to build extensible and lightweight applications. It enables developers to discover and implement extensions without the need for configuration. MEF also helps extension creators in encapsulating their code to avoid hard dependencies. Additionally, it enables the reuse of extensions not just within a single application but across multiple applications.

Composition container

At the heart of the MEF composition model lies the composition container, serving as the repository for all available parts and leading their composition. Composition is the matching up of imports to exports.

Imports and Exports

⁴Source: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/media/syntax-visualizer/visualize-csharp.png> (2024-01-07)

Listing 12: ICustomHandler for MEF example

```
1 public interface ICustomHandler {  
2     string Handle(string input);  
3 }
```

First an interface needs to be designed.

Listing 13: CustomHandler Export for MEF example

```
1 [Export(typeof(ICustomHandler))]  
2 class CustomHandler : ICustomHandler {  
3 }
```

For a successful match between an import and an export, both must share the same contract. If an export is associated with a different contract, such as `typeof(CustomHandler)`, it won't match an import looking for a different contract, leading to the import not being fulfilled. The contract must be an exact match.

Listing 14: ICustomHandler Import for MEF example

```
1 [Import(typeof(ICustomHandler))]  
2 public ICustomHandler customHandler;
```

In MEF, every import is associated with a contract that determines which exports it matches with. This contract can be an explicit string or automatically generated based on a type, like the `ICustomHandler` interface in this context. Any export that declares a matching contract can fulfill this import. The contract is not necessarily tied to the type of the importing object. Even though the object type might be `ICustomHandler`, the contract can differ. By default, MEF uses the type of the import to determine the contract, unless explicitly specified otherwise.

*✗ Abgrenzung zu anderen DI-Containern. Was bietet MEF zusätzlich?
Wenn ist MEF für die Arbeit relevant.*

6 Building the Visual Studio Extension

6.1 Creating a new Extension Project

6.2 VS Commands

6.3 Custom windows

6.4 MEF Extension

6.5 Code Analyser

7 CLOSING

7 Closing

REFERENCES

References

- Agnihotri, A., Gautam, H., and Singh, H. (2022). Discord bot.
- Developer.com (2012). An introduction to microsoft roslyn.
- Discord (2022). What is discord?
- Discord.Net (2015). Discord.net.
- GameInfoHub (2023). Inside the discord revolution: Insights into gaming's powerful communication platform.
- Microsoft (2021a). Understand the .net compiler platform sdk model.
- Microsoft (2021b). Visual studio extensibility cookbook.
- Microsoft (2021c). Work with a workspace.
- Microsoft (2021d). Work with semantics.
- Microsoft (2021e). Work with syntax.
- Microsoft (2022). Explore code with the roslyn syntax visualizer in visual studio.
- Microsoft (2023). Visual studio extension.
- TheEnlightenedMindset (2023). Who invented discord? the story of its innovators and developers.
- ToptalDevelopers (2022). How to make a discord bot: an overview and tutorial.
- Vasani, M. (2017). *Roslyn Cookbook*. Packt Publishing Ltd.

Groß- u. Kleinschreibung wie im zitierten Werk
Ausführliche bibliografische Angaben: Autor, ... Mindestanfor-
derung: Man muss die zitierte Stelle finden können. Sie zitieren
fast ausschließlich Software-Doku-
mentationen.
Bei umfangreichen Quellen muss
man beim Zitat auch angeben, wo
man die zitierte Passage in der
Quelle findet. z.B. [Xxx2023, pp. 10-12]